GUÍA PRÁCTICA PARA PROGRAMADORES

3 tareas que probablemente sigues haciendo a mano y que puedes automatizar hoy mismo

Code review, documentación y tests: qué herramientas de IA usar para cada una, para qué sirve cada una, cómo se instalan y cómo se usan, paso a paso.

Code review | Documentación | Tests

01 Code review

En lugar de revisar cada Pull Request desde cero, puedes automatizar este proceso. Estas herramientas analizan tus PRs y dejan comentarios directamente sobre el código.

Greptile

`greptile.com`

Agente de IA que revisa automáticamente cada pull request con comprensión completa de tu base de código. A diferencia de un linter tradicional, que analiza archivos por separado, Greptile construye un grafo de todo tu repositorio (funciones, clases y dependencias) para entender cómo un cambio afecta al resto del sistema. Publica los hallazgos como comentarios en el PR en unos 3 minutos, con sugerencias de arreglo con un clic.

INSTALACIÓN (GITHUB)

1. Entra en `app.greptile.com/login` y crea una cuenta con email, Google, GitHub o GitLab.
2. Ve a la sección **Code Providers** y pulsa **Connect GitHub Cloud** (o **Add Provider**).
3. Elige la cuenta u organización de GitHub donde quieres instalar **Greptile Apps**.
4. Concede acceso a los repositorios: a todos (actuales y futuros) o solo a los que selecciones.
5. En `app.greptile.com/review/github`, activa la revisión automática para los repositorios que quieras que Greptile revise.

CÓMO SE USA

1. Abre un Pull Request como siempre en GitHub o GitLab.
2. Greptile analiza los cambios con el contexto completo del repo y publica un resumen del PR junto con comentarios en las líneas donde detecta problemas.
3. Aplica las sugerencias con un clic o responde a los comentarios para pedir aclaraciones.
4. Con el tiempo, Greptile lee los comentarios que tu equipo deja en PRs anteriores y ajusta qué tipo de hallazgos prioriza.

CodeRabbit

`coderabbit.ai`

Revisor de código con IA que se instala como app nativa de GitHub (también soporta GitLab, Bitbucket y Azure DevOps) y revisa automáticamente cada pull request, dejando un resumen y comentarios en línea sobre errores de lógica, vulnerabilidades de seguridad, rendimiento y estilo. También tiene una extensión de IDE para revisar tu código antes incluso de abrir el PR.

INSTALACIÓN (GITHUB APP)

1. Entra en `coderabbit.ai` y crea una cuenta con tu cuenta de GitHub.
2. Autoriza la app **coderabbitai** cuando GitHub te pida permisos (lectura del código, escritura en comentarios de PR).
3. Elige si das acceso a todos los repositorios o solo a algunos seleccionados, y confirma la instalación.
4. Opcional: añade un archivo `.coderabbit.yaml` en la raíz del repo para configurar el comportamiento de revisión (ramas objetivo, qué bloquea el merge, instrucciones en lenguaje natural).

INSTALACIÓN (EXTENSIÓN DE IDE, OPCIONAL)

1. En VS Code (o un fork compatible como Cursor o Windsurf), abre **Extensiones** y busca `coderabbit`.
2. Pulsa **Install** e inicia sesión con tu cuenta de CodeRabbit cuando te lo pida.

CÓMO SE USA

1. Abre un Pull Request en un repositorio con CodeRabbit instalado.
2. En unos minutos, CodeRabbit publica un resumen de los cambios y comentarios en línea sobre los problemas que encuentra.
3. Puedes mencionar a `@coderabbitai` en un comentario del PR para pedirle que revise de nuevo o resuelva dudas.
4. Desde el plan Pro, muchas sugerencias incluyen un cambio propuesto que aplicas directamente desde GitHub con un clic.
5. Con la extensión de IDE, abre el panel de CodeRabbit y pide una revisión manual de tu rama antes de subir el PR.

02 Documentación

Mantener la documentación actualizada después de cada cambio es una tarea que casi todos posponemos. Estas dos herramientas la automatizan de formas distintas.

Swimm

`swimm.io`

Conecta la documentación directamente con tu código, a diferencia de un wiki o un README estático. Los documentos de Swimm se guardan como Markdown dentro de tu propio repositorio y quedan enlazados a funciones o bloques de código concretos: si el código cambia, Swimm detecta que la documentación quedó desactualizada y te avisa para corregirla.

INSTALACIÓN

1. Entra en `swimm.io` y crea una cuenta.
2. Conecta tu repositorio (GitHub, GitLab o Bitbucket) desde el asistente de configuración.
3. Instala el plugin de Swimm para tu editor: en VS Code, busca `Swimm` en Extensiones y pulsa **Install** (también hay plugin para la familia JetBrains).
4. Inicia sesión en el plugin con tu cuenta de Swimm y sincroniza el repositorio.

CÓMO SE USA

1. Desde el editor, crea un documento nuevo con el editor de texto enriquecido de Swimm.
2. Selecciona fragmentos de código de tus archivos abiertos y añádelos al documento como snippets vivos.
3. Al navegar por tu código verás un icono junto a las líneas que tienen documentación asociada; haz clic para leerla sin salir del IDE.
4. Si refactorizas el código documentado, Swimm marca el snippet como desactualizado y te guía para actualizar la selección antes del commit.

Mintlify

`mintlify.com`

Genera documentación técnica (guías, referencia de API, changelog) a partir de tu código y tu especificación OpenAPI, y la publica como un sitio de documentación con IA integrada capaz de responder preguntas citando tus propios documentos. También permite generar documentación de un fragmento de código seleccionado directamente en el editor.

INSTALACIÓN (CLI)

1. Instala Node.js 20.17 o superior si no lo tienes.
2. Instala la CLI con `npm i -g mint` (o `pnpm add -g mint`).
3. Clona la plantilla inicial con `git clone https://github.com/mintlify/starter.git`, o conecta tu propio repositorio desde el dashboard de Mintlify.
4. Dentro de la carpeta del proyecto, ejecuta `mint dev` para previsualizar la documentación en local.

GENERAR DOCUMENTACIÓN DESDE UN REPO EXISTENTE

1. Entra en `mintlify.com` y conecta tu repositorio de GitHub.
2. Mintlify lee tu base de código y genera automáticamente una primera versión de la documentación (guías y referencia de API si detecta una spec OpenAPI).
3. Revisa el resultado: cualquier ajuste se guarda como un PR que fusionas cuando estés conforme.

CÓMO SE USA

1. Edita el contenido desde el editor web de Mintlify o directamente en los archivos `.mdx` de tu repositorio.
2. Con la extensión de editor, selecciona un bloque de código y usa la opción **Generate Docs** para que Mintlify escriba la documentación de esa función.
3. Configura automatizaciones (por ejemplo, generar documentación en cada push) para mantener los docs sincronizados sin intervención manual.
4. Cada cambio fusionado en la rama principal se despliega automáticamente en tu sitio de documentación.

03 Tests

Puedes hacer que la IA genere buena parte del trabajo inicial de testing por ti. Estas herramientas generan y revisan tests; la última es específica para Java.

Qodo

`qodo.ai`

Extensión de IDE (antes CodiumAI) que analiza el comportamiento de una función —tipos de entrada, ramas condicionales— y genera tests unitarios con casos límite incluidos. También ofrece revisión de código local y chat con contexto del proyecto.

INSTALACIÓN

1. Abre VS Code y ve a **Extensiones** (`Ctrl+Shift+X` / `Cmd+Shift+X`).
2. Busca `Qodo` y pulsa **Install** (también disponible para IntelliJ, PyCharm y el resto de IDEs JetBrains desde Settings > Plugins).
3. Reinicia el editor y crea una cuenta o inicia sesión cuando te lo pida. El plan gratuito incluye 250 créditos al mes, sin tarjeta de crédito.

CÓMO SE USA

1. Abre el archivo con la función que quieres testear y coloca el cursor dentro de ella (o selecciónala).
2. Abre el panel de chat de Qodo y escribe `/test`, o haz clic derecho y elige la opción de generación de tests del menú contextual.
3. Qodo analiza la función y te propone un archivo de test; revísalo y acéptalo para guardarlo en tu proyecto.
4. Usa los tests generados como punto de partida: son sólidos para lógica unitaria, pero los flujos de integración o los que dependen de base de datos suelen necesitar ajuste manual.

TestSprite

`testsprite.com`

Agente de testing autónomo que explora tu aplicación en funcionamiento (web o API), entiende cómo se comporta y genera y ejecuta un plan de pruebas completo sin que tengas que escribir scripts ni selectores. Se conecta a tu editor mediante un servidor MCP y ejecuta las pruebas en la nube.

INSTALACIÓN (SERVIDOR MCP)

1. Necesitas Node.js 22 o superior y una cuenta gratuita en `testsprite.com`.
2. Instala el paquete con `npm install -g @testsprite/testsprite-mcp@latest`.
3. Añade la configuración del servidor MCP a tu editor (Cursor, Claude Code, Windsurf, VS Code o GitHub Copilot), con tu API key en la variable de entorno `API_KEY`.
4. Reinicia el editor y comprueba que las herramientas de TestSprite aparecen disponibles para el asistente de IA.

CÓMO SE USA

1. Arranca tu aplicación en local (por ejemplo, `npm run dev` para el frontend o `node index.js` para el backend).
2. En el chat de tu editor, escribe algo como "Ayúdame a testear este proyecto con TestSprite" (puedes arrastrar la carpeta del proyecto o pedir algo más específico, como probar el flujo de checkout).
3. TestSprite lee tus requisitos (o los infiere del código), genera un plan de pruebas, lo ejecuta en la nube y te devuelve un informe con los fallos encontrados.
4. Si tu asistente de IA tiene permiso, puede aplicar arreglos automáticos a partir de los bugs detectados.

RoostGPT

`roost.ai`

Extensión que genera tests unitarios, de API, funcionales y de UI para varios lenguajes —Java, Python, Go, Node.js y C#— directamente desde el explorador de archivos del editor, sin necesidad de seleccionar código manualmente.

INSTALACIÓN

1. Abre VS Code, ve a Extensiones y busca `Roost GPT` (o instálala desde el Marketplace de Visual Studio Code).
2. Abre el panel de RoostGPT en la barra de actividad e introduce tu clave de licencia. Puedes conseguir una gratis registrándote con tu email de empresa en `roost.ai`.
3. Configura el dominio de Roost (por defecto `app.roost.ai`) y el lenguaje de tu proyecto desde los ajustes de la extensión.

CÓMO SE USA

1. En el explorador de archivos de VS Code, haz clic derecho sobre el archivo o carpeta que quieres testear.
2. Elige el tipo de test: unitario, funcional, de integración o de API. Para API, haz clic derecho sobre tu colección de Postman o tu spec de Swagger/OpenAPI.
3. Selecciona el framework de testing en el menú emergente y confirma.
4. RoostGPT genera los tests y los guarda en la carpeta estándar de test de tu proyecto, respetando la estructura del código fuente.

Diffblue Cover

`diffblue.com`

Herramienta específica para Java que genera tests unitarios analizando el bytecode compilado de tu proyecto, no solo el texto del código. Usa aprendizaje por refuerzo para crear, ejecutar y ajustar cada test hasta maximizar la cobertura, siguiendo el patrón arrange-act-assert.

INSTALACIÓN

1. Asegúrate de que tu proyecto compila y pasa sus tests actuales: Java 8, 11, 17 o 21 (o Kotlin), con Maven 3.2.5+ o Gradle 4.9+.
2. En IntelliJ IDEA, ve a **Settings/Preferences > Plugins > Marketplace** y busca `Diffblue Cover`.
3. Instala la Community Edition, gratuita para proyectos individuales, y reinicia IntelliJ.
4. Activa tu licencia desde el menú **Diffblue > Activate License**.

CÓMO SE USA

1. Compila tu proyecto primero (por ejemplo, desde el panel de Maven, doble clic en `package`), porque Diffblue analiza el bytecode.
2. Verás un icono de matraz junto a cada clase y método que se puede testear.
3. Haz clic en el icono de un método o clase concreto, o haz clic derecho sobre un archivo o carpeta completa y elige **Write Tests** para generar tests a mayor escala.
4. Diffblue crea los tests, los ejecuta y ajusta los candidatos automáticamente hasta encontrar el conjunto que maximiza la cobertura, y los añade a tu carpeta `src/test/java`.

> Java es el único lenguaje que soporta Diffblue Cover por ahora. Para el resto de lenguajes de esta guía, usa Qodo, TestSprite o RoostGPT.

Elige una tarea y automatízala esta semana

No hace falta adoptar las ocho herramientas a la vez. Elige la tarea que más tiempo te quita ahora mismo —code review, documentación o tests— y prueba una de las opciones de esa sección en tu próximo proyecto.